

Linux Server Troubleshooting Reference

This reference guide was put together during my Linux learning journey as a practical troubleshooting resource. Not every command here is one I fully understand yet — this is a living document that will grow alongside my skills. The flowchart covers the most common problems a sysadmin encounters, the commands used to investigate them, and the thought process for working through an unknown issue systematically.

The golden rule: understand before you fix.

The Troubleshooting Flowchart

When a server problem is reported, always start with initial checks before diving into the specific symptom. This gives you a baseline picture of the system's health.

Step 1 — Initial Checks (always run these first)

whoami — confirm who you are and what privileges you have

w — see who else is logged in and what they are doing

uptime — how long the system has been running and current load

df -h — quick overview of disk space across all filesystems

free -h — current RAM and swap usage

Step 2 — Identify the Symptom Category

Based on what the initial checks show (and what the person reporting the problem describes), follow the relevant branch below.

Service Down

Someone can't reach a web app, database, or other service that was working before.

1. Check the service status

```
systemctl status servicename
```

Shows whether the service is running, stopped, or failed. The last few log lines often tell you why it failed.

2. Read the logs

```
journalctl -u servicename --since "1 hour ago"
```

Shows the service's log output. Look for error messages — these usually point directly at the cause.

3. Check for port conflicts

```
sudo ss -tlnp
```

If a service failed to start because another process is already using its port, this will show you what is occupying it.

4. Restart the service (once you understand why it stopped)

```
sudo systemctl restart servicename
```

Only restart once you understand the cause — restarting a broken service without fixing the root cause just delays the problem.

Common causes of service failures

- Crashed unexpectedly — check journalctl for error messages
- Port conflict — another process grabbed the same port (check ss -tlnp)
- Configuration change broke something — check recent changes to config files
- Disk full — service can't write logs or data (check df -h)
- SSL/TLS certificate expired — check cert expiry dates
- Dependency service down — e.g. database stopped, web app can't start

Network Issues

Can't reach the server, or the server can't reach the internet or other services.

1. Check network interfaces

```
ip a
```

Shows all network interfaces, their IP addresses, and whether they are UP or DOWN. If the relevant interface is DOWN, that's your problem.

2. Test basic connectivity

```
ping -c 4 8.8.8.8
```

Sends 4 packets to Google's DNS server. If these fail, the server has no internet access. The -c flag limits to 4 packets (without it, ping runs forever).

3. Check what is listening on which ports

```
sudo ss -tlnp
```

Shows every TCP port the server is listening on, and which process owns it. Useful for confirming a service is actually running and accepting connections.

4. Check routing

```
ip route
```

Shows the routing table — the rules the system uses to decide where to send traffic. The 'default via x.x.x.x' line is the gateway (usually your router). If this is missing, the server has no route to the internet.

Disk Space Issues

Services failing to write logs, databases crashing, or the system behaving strangely can all be caused by a full disk.

1. Check filesystem usage

```
df -h
```

Shows all filesystems and their usage percentage. Look for anything at or near 100% — that's your problem filesystem.

2. Find what is taking up space

```
du -sh /*
```

Shows the size of each top-level directory. Drill into the biggest one to find the culprit. For example, if /var is large:

```
du -sh /var/*
```

3. Check log file sizes

```
ls -lh /var/log
```

Overgrown log files are a very common cause of full disks on servers. Look for any file that is unexpectedly large.

4. Clean up unused packages

```
sudo apt autoremove
```

Removes packages that were installed as dependencies but are no longer needed. Safe to run, often frees up significant space.

Performance Issues

Server is running slowly, timing out, or struggling under load.

1. Check CPU and memory usage

```
htop
```

Interactive real-time view of all processes, CPU per core, RAM, and swap. Press M to sort by memory, P to sort by CPU. Press Q to quit.

2. Find the top CPU-consuming processes

```
ps aux --sort=-%cpu | head -10
```

Shows the top 10 processes by CPU usage. The --sort=-%cpu flag sorts highest first.

3. Check load average

```
uptime
```

The three numbers at the end are load averages over 1, 5, and 15 minutes. A load average consistently higher than your number of CPU cores suggests the system is overloaded.

4. Kill or restart a runaway process

```
kill PID
```

Replace PID with the process ID from htop or ps aux. If the process won't stop:

```
kill -9 PID
```

The -9 flag force-kills the process immediately. Use as a last resort.

Security Concerns

Unexpected behaviour, unknown processes, or suspicious login activity.

1. Check who is currently logged in

```
w
```

Shows all currently active sessions, where they connected from, and what they are doing. On Ubuntu 26.04, use `loginctl list-sessions` as an alternative (the `who` command does not work on this version due to the Y2038 fix).

2. Check login history

```
last
```

Shows a history of logins — who connected, from where, when, and for how long. Look for logins at unusual times or from unfamiliar IP addresses.

3. Look for unusual processes

```
ps aux
```

Shows all running processes from all users. Look for anything you don't recognise, especially processes running as root.

4. Check SSH authentication logs

```
journalctl -u ssh --since "24 hours ago"
```

Shows SSH login attempts over the last 24 hours. A large number of failed login attempts from external IPs indicates a brute-force attack (common on internet-exposed servers, less likely on a home server behind a router).

5. Check open connections and listening ports

```
sudo ss -tlnp
```

Look for anything listening on an unexpected port, or any process you don't recognise owning a port.

Quick Command Reference

A summary of all commands covered in this guide.

System information

whoami — show current username

w — show currently logged-in users and activity

last — show login history

uptime — system uptime and load averages

hostname — show server name

cat /etc/os-release — show OS version and codename

hostnamectl — show OS, kernel, and hardware details

nproc — show number of CPU cores

Disk

df -h — show filesystem disk usage (human-readable)

du -sh /path — show disk usage of a specific directory (summarised)

du -sh /* — show disk usage of all top-level directories

ls -lh /var/log — show log files with sizes

sudo apt autoremove — remove unneeded packages

Memory

free -h — show RAM and swap usage (human-readable)

Network

ip a — show network interfaces and IP addresses

ip route — show routing table and default gateway

ping -c 4 8.8.8.8 — test internet connectivity (4 packets)

sudo ss -tlnp — show listening TCP ports and owning processes

Processes

ps aux — show all running processes from all users

ps aux --sort=-%cpu | head -10 — top 10 processes by CPU usage

htop — interactive real-time process and resource viewer

kill PID — terminate a process by its ID

kill -9 PID — force-kill a process that won't stop normally

Services

systemctl status name — check if a service is running or failed

sudo systemctl start name — start a service

sudo systemctl stop name — stop a service

sudo systemctl restart name — restart a service

sudo systemctl enable name — start service automatically at boot

sudo systemctl disable name — prevent service starting at boot

journalctl -u name — show logs for a specific service

journalctl -u name --since "1 hour ago" — logs from the last hour only

The Golden Rules

- Establish the basics first — who are you, who else is on the system, how long has it been running
- Read the logs before touching anything — they usually tell you exactly what went wrong
- Understand before you fix — restarting a broken service without fixing the root cause just delays the problem
- Confirm the fix worked — after any change, verify the service is running and the problem is resolved
- Ask before assuming — if you're on someone else's server, check which services should be running before stopping anything

This guide will be updated as new commands and scenarios are encountered. Last updated June 2026.